

Project [GEO CRM]: Technical Specification & Documentation

Status: Draft / In-Progress

Version: 1.0.0

Developers: [Malachi Egbugha (Backend), Tochukwu Orizu (Frontend)]

UI/UX: Egwuchukwu Nwizuzu

DevOps: Abiola Babatunde

Infrastructure:

Security:

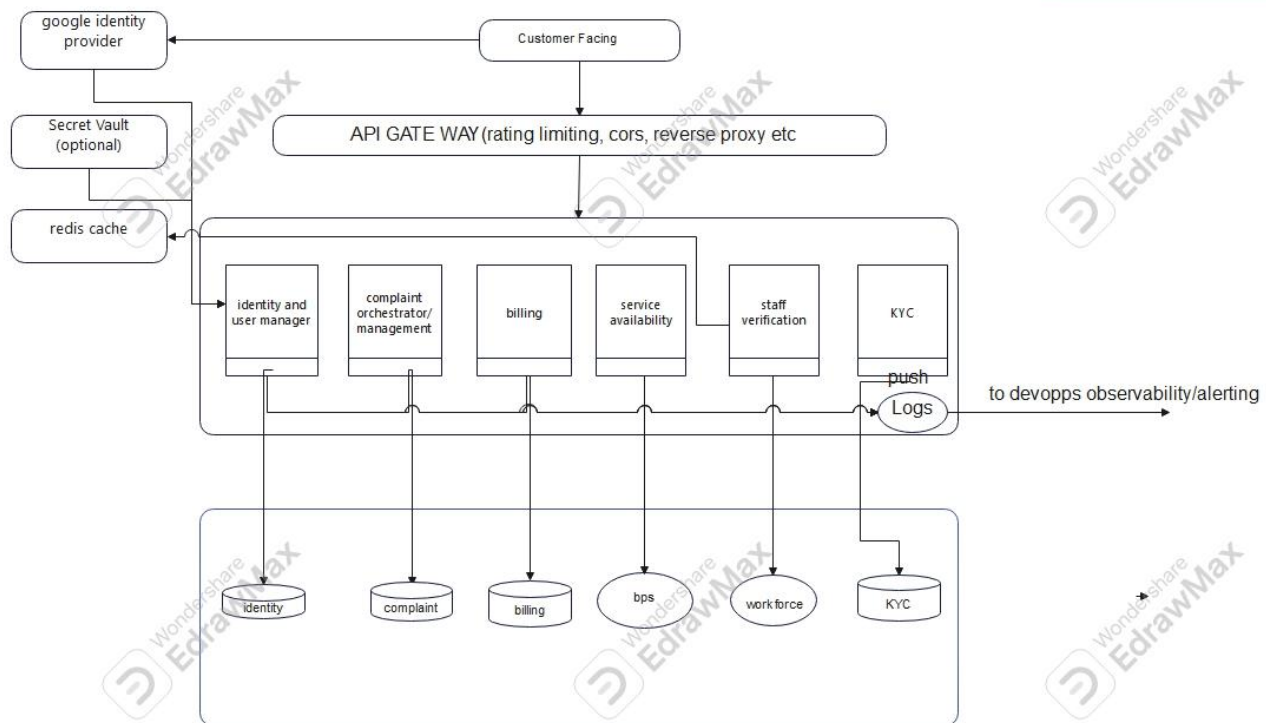
Date: May 2026

1. System Overview

This document provides a comprehensive technical description of the CRM platform architecture. The system is designed as a cloud-native, micro services-based solution that handles customer-facing interactions, internal workforce management, billing, complaint resolution, and service availability monitoring. In addition to the baseline architecture, this document details the integration of an AI Customer Agent — a conversational AI layer that augments customer self-service and automates first-line support workflows.

It is designed as a modular, containerized application prioritizes scalability, security, and observability, leveraging an API Gateway, distributed micro services, a Redis caching layer, and a dedicated logging pipeline that pushes to DevOps observability and alerting tools.

2. System Architecture



The CRM system is structured into four logical tiers:

- **Presentation / Access Tier** — Customer-facing entry point and authentication
- **Gateway Tier** — API Gateway handling cross-cutting concerns
- **Micro services Tier** — Domain-specific business logic services
- **Data Tier** — Isolated data stores for each service

An AI Customer Agent sits at the presentation tier, intercepting and resolving routine queries before they propagate to backend services. All micro services emit structured logs that are forwarded to a central DevOps observability and alerting stack.

Tier	Key Components
Presentation	Customer Facing App, Google Identity Provider
AI Agent	AI Customer Agent (NLP, Dialogue Mgmt, Tool Calls)
Gateway	API Gateway (Rate Limiting, CORS, Reverse Proxy), Redis Cache, Secret Vault
Microservices	Identity & User Mgr, Complaint Orchestrator, Billing, Service Availability, Staff Verification
Data	Identity DB, Complaint DB, Replication Arms, BPS DB, Workforce DB

3. Functional Requirements

3.1 Identity and Access Management

- **External Authentication:** Support for Google Identity Provider for customer-facing authentication.
- **Staff Verification:** Integration with Azure AD for internal workforce authentication and authorization.

3.2 Core CRM Services

- **Complaint Orchestration:** Automated management and routing of customer complaints.
- **Billing & Payments:** Handling of financial transactions and history.
- **Service Availability:** Real-time tracking of service status for customers.
- **Staff Verification:** A dedicated module to validate internal credentials and permissions.

3.3 Communication & Data

- **Asynchronous Messaging:** Use of a Message Broker for decoupled communication between services.
- **Data Persistence:** Service-specific databases including Identity, Complaint, Billing, and BPS stores.

4. Non-Functional Requirements

3.1 Security & Traffic Management

- **API Gateway:** Enforcement of Rate Limiting, CORS policies, and Reverse Proxying to protect backend services.
- **Secret Management:** Optional integration of a Secret Vault to manage sensitive credentials and API keys.

3.2 Performance & Scalability

- **Modularity:** Services are separated to allow independent scaling based on demand (e.g., scaling the Billing service during peak payment periods).
- **Asynchronous Processing:** Staging databases and brokers ensure the system remains responsive under heavy load.
- **Caching with Redis:** A shared Redis cache layer sits between the micro services and their underlying data stores, dramatically reducing database round-trips for frequently accessed data such as session tokens. This keeps response times consistently low even as concurrent user load increases.

3.3 Observability

- **Centralized Logging:** All services must output logs to a unified stream.
- **Real-time Alerting:** Automated "push" of logs to a DevOps observability platform for immediate alerting on system failures.

5.0 Component Descriptions

5.1 Customer Facing Layer

The customer-facing interface is the primary entry point to the platform. It can be a web application, mobile application. Authentication is delegated to Google Identity Provider via OAuth 2.0 / OpenID Connect, ensuring that credentials are never stored directly within the CRM platform. Upon successful authentication, a signed JWT is issued and passed in the Authorization header of downstream requests.

5.2 Google Identity Provider

Google Identity Provider (IdP) serves as the federated authentication authority. The customer-facing layer redirects unauthenticated users to Google for sign-in. The platform receives an ID token which is validated by the Identity and User Manager microservice on every request. This eliminates password management overhead within the CRM and benefits from Google's advanced threat detection.

5.3 Secret Vault (Optional)

A Secret Vault component — marked optional in the architecture — is used to securely store and retrieve sensitive configuration values such as database credentials, API keys, third-party service tokens, and encryption keys. Services retrieve secrets at startup or on demand through a secure API call, avoiding hard-coded credentials in application code or environment variables. This component is recommended for production deployments.

5.4 API Gateway

The API Gateway is the single entry point for all authenticated traffic. It enforces the following cross-cutting concerns:

- Rate Limiting — Prevents abuse by capping requests per user/IP per time window
- CORS — Enforces Cross-Origin Resource Sharing policies for browser-based clients
- Reverse Proxy — Routes requests to the appropriate downstream microservice
- Request Validation — Validates JWT tokens and rejects malformed requests early
- TLS Termination — Handles HTTPS termination, forwarding plain HTTP internally

The gateway decouples clients from the internal service topology, allowing services to be scaled, versioned, or replaced without impacting client integrations.

5.5 Redis Cache

Redis acts as a shared, in-memory caching layer accessible by multiple micro services. It is used primarily to cache session data, frequently queried reference data, and rate-limit counters. All micro services that benefit from reduced database round-trips read from and write to the Redis cluster. Cache invalidation strategies (TTL-based and event-driven) are defined per service domain.

5.6 Micro services

Service	Responsibility
Identity & User Manager	Manages user profiles, roles, permissions, and session validation. Validates JWT on every request and enforces access control policies.
Complaint Orchestrator / Management	Receives, routes, and tracks customer complaints through their full lifecycle. Coordinates with other services for resolution.
Billing	statement, payment status, and billing history. Integrates with the BPS (Billing Processing System) data store.
Service Availability	Monitors and exposes the real-time availability status of platform services.
Staff Verification	Verifies staff credentials, roles, and assignments. Ensures only authorised personnel can perform privileged operations.

5.7 Data Tier

Each micro service owns its data store in accordance with the database-per-service pattern, which prevents tight coupling and enables independent scaling.

5.8 Logging & Observability

All micro services emit structured logs in a common format. A central log aggregation component collects these logs and pushes them to the DevOps Observability and Alerting stack.

6. AI Customer Agent

6.1 Overview: The AI Customer Agent is a large language model (LLM)-powered assistant that operates directly within the customer-facing channel. Its responses are governed by a structured knowledge base. When a customer submits a query, the agent retrieves the most relevant knowledge base entries and formulates a response from that grounded context — it does not

speculate beyond what the knowledge base contains. For queries outside its knowledge scope, the agent escalates to a human agent with full conversation context intact.

6.2 Core Capabilities

- **Knowledge Base Retrieval (RAG)** — Performs semantic search against the internal knowledge base to surface the most relevant content before composing a response
- **Natural Language Understanding (NLU)** — Parses and interprets free-text customer queries in real time, mapping them to known topics and intents
- **Grounded Response Generation** — Constructs replies strictly from retrieved knowledge base content, citing the source article or policy where applicable
- **Intent Classification** — Identifies the customer's intent (e.g. check bill, raise complaint, check service status) and routes to the appropriate knowledge domain
- **Dialogue Management** — Maintains multi-turn conversation context, enabling coherent follow-up handling without repeating retrieved context on every turn
- **Escalation Handling** — Detects when no knowledge base match meets the confidence threshold and routes the customer to a human agent with the full session transcript
- **Sentiment Analysis** — Monitors customer sentiment throughout the conversation and triggers proactive escalation when sustained frustration is detected
- **Audit Logging** — All agent responses, retrieved knowledge base references, and escalation events are logged to the observability stack with the user identity and a correlation ID

7 Security Architecture

Security is enforced at every tier of the system:

- **Authentication** — Federated via Google IdP using OAuth 2.0 / OpenID Connect. No passwords are stored in the CRM platform.
- **Authorization** — JWT claims are validated by the Identity & User Manager on every request. Role-Based Access Control (RBAC) is enforced at service level.
- **Secret Management** — Sensitive configuration is stored in the Secret Vault, never in source code or environment variables.
- **Transport Security** — All external communication is TLS 1.2+ encrypted. Internal service-to-service communication may use mTLS.
- **Caching Security** — Redis instances are password-protected and network-isolated within the service tier.
- **Rate Limiting** — The API Gateway enforces per-user and per-IP rate limits to mitigate abuse and DDoS attacks.

8. Observability & Alerting

All micro services and the AI Customer Agent push structured logs to the central logging pipeline. Log records include a service identifier, timestamp, correlation ID, user identity hash, log level, and event payload. The application will be developed to output logs directly to **Standard Output (stdout)** in **JSON format**. This eliminates the need for managing log files inside the container and allows the Docker logging driver to handle the stream.

9. Deployment: The CRM application utilizes a **Standalone Container Architecture**. This approach prioritizes simplicity and performance by running services directly on the host machine without the overhead of a complex orchestrator.

- **Containerization:** The application is packaged using Docker, ensuring that the development environment is identical to the production environment.
- **Service Management:** We use **Docker Compose** to define and run the multi-container application. Services are configured with a restart: always policy to ensure they recover automatically after a host reboot or unexpected crash.